

Towards learning-based trajectory tracking control for a planetary exploration rover: Adaptive model predictive control

Niklas Baldauf¹, Kristin Lakatos², Alexander Meinert¹ and Alen Turnwald¹

Abstract—Autonomous Robotic vehicles are important for exploring distant planets due to the risks associated with manned missions. However, they face challenges in navigating unknown terrain, requiring a deep understanding of wheel-ground interaction for effective slip estimation and compensation. Conventional methods have proven insufficient in addressing these challenges. The DeLeMIS project showcases AI methods to enhance autonomy in uncertain terrain, focusing on the trajectory tracking performance. This paper presents the development and evaluation of an adaptive model predictive control algorithm within the project, extended by machine-learning techniques. The learning-based algorithm and conventional baseline controllers are implemented and systematically evaluated through a comprehensive test campaign, which is described in our previous publication. This paper gives an overview of the control architecture and the training process, providing detailed insights into the neural network architecture and the results on a prototypic rover hardware.

Index Terms—planetary exploration rover, model predictive control, learning-based control, machine learning

I. INTRODUCTION

Throughout history, humanity has been driven by the ambition to explore distant planets. Today, both manned and unmanned missions are deployed to investigate nearby celestial bodies. While manned missions pose significant risks, robotic vehicles have become vital for exploration [1]–[3]. Yet, ground-based vehicles face challenges, notably unpredictable terrain. Consequently, understanding wheel-ground interaction is crucial, particularly in slip estimation and compensation.

Addressing these challenges, the ESA founded project DeLeMIS showcases AI methods to improve the autonomy in uncertain terrain, even with limited prior information about the terrain properties. The project focuses on testing and comparing learning-based control algorithms to address motion control complexities in unpredictable environments. In particular, a trajectory tracking algorithms utilizing machine learning is implemented in simulation and on a rover prototype. A comprehensive test campaign is performed to evaluate their performance systematically. Hardware tests are conducted on the Lightweight Rover Unit (LRU) at indoor test facilities of the German Aerospace Center (DLR).

The aim of our work is to improve trajectory tracking capabilities in uncertain terrain. This method particularly focuses on enhancing the lateral and longitudinal deviations from a predefined target trajectory. Therefore, a combination of model

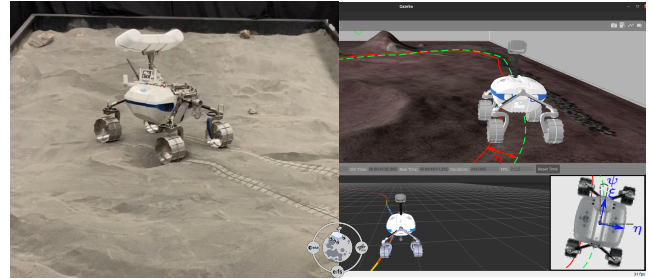


Fig. 1: The LRU driving in the Planetary Exploration Lab (PEL) during the test campaign (left). The Gazebo simulation including the LRU in a modeled landscape [11] (right). The simulation is used for software in the loop tests.

predictive control (MPC) with learning-based extensions is implemented and tested. A comprehensive overview of such extensions is provided in [4] and [5]. Notably, our approach requires minimal training data and excludes Gaussian processes, setting it apart from related methods as presented in [6] or [7]. Moreover, our approach can also be used beyond lap-based scenarios as in [8]. A comparable online learning approach was presented in [9]. However, our work comprises an extensive test campaign on a real rover prototype in a space-analog environment.

Building upon a detailed description of the performed tests already published in our previous work [10], this paper shows the development of learning-based adaptive MPC for trajectory tracking. Detailed insights to the neural net architecture are presented. Finally, results of the algorithm implemented on a prototype rover hardware are shown and discussed.

The paper is organized as follows: The test scenario and problem statement is briefly outlined in Sec. II. The used controllers are shown in Sec. III. Some selected results of the test campaign are shown in Sec. V. A discussion of the experimental setup and the results follows in Sec. VI. Sec. VII concludes the paper.

II. SCENARIO AND TEST SETUP

The goal of the performed test campaign is to demonstrate and evaluate improvement potentials of learning algorithms to the rover's trajectory tracking behavior. Thus, the test scenarios are defined to be implemented in a space-analog test environment and use real hardware.

As a preparation for hardware tests, all software modules are initially tested in a Gazebo simulation, see Fig. 1. Afterwards, hardware test are performed with the Lightweight Rover Unit at indoor test facilities located at the DLR Oberpfaffenhofen.

¹Niklas Baldauf, Alexander Meinert and Alen Turnwald are with the department of Space Applications (e:space), e:fs TechHub GmbH. {Niklas.Baldauf, Alen.Turnwald, Alexander.Meinert}@efs-techhub.com

²Kristin Lakatos is with the Institute of Robotics and Mechatronics, German Aerospace Center (DLR). Kristin.Lakatos@dlr.de

The LRU is a terrestrial prototype of a planetary exploration rover with four individually steerable wheels [12], [13]. An impression of the rover driving in the testbed filled with a Moon-analog soil is depicted in Fig. 1. A detailed description of the rover hard- and software and the test facilities as well as the test procedure is given in our previous publication [10].

To provide a framework for a systematic analysis of the test results, the investigated control algorithms are kept as isolated as possible. The focus of this work is the development and evaluation of the *motion controller* subsystem, which is situated between the planning layer and the low-level motor control layer. Its primary function is to translate the high-level trajectory instructions from the planning layer into rover commands while accounting for uncertainties and constraints. The low-level motor control layer is then tasked with executing the commanded actions by controlling the individual actuators. The fundamental task of the motion control subsystem is to ensure that the rover adheres closely to the prescribed trajectory, even when the terrain is only partially understood or entirely unknown. Thereby, the objective is that the performance of this subsystem improves over time through iterative refinement. The continual enhancement serves as an indicator of the system's learning capabilities and its ability to navigate more effectively in varying environments.

In order to ensure an isolated evaluation of the selected algorithms, the tests are conducted under the following conditions: First, the control algorithms use the position measurements of a ground truth positioning system installed within the testing environment. The measurements thus provide accurate positional information without introducing biases or inaccuracies introduced by an erroneous self-localization, which could affect the evaluation of learning algorithms. Second, the trajectories provided to the motion control subsystem are always assumed to be reachable with no impassable obstacles. Note that these prerequisites do only apply to the setup of the test campaign, in order to ensure comparability and significance of the obtained results. The tested learning-based algorithms do not rely on any of these requirements.

III. PROBLEM STATEMENT

A controller architecture combining a non-linear Lyapunov controller (NLC) and a layered model predictive controller is employed for trajectory reference tracking. Detailed descriptions of this method can be found in [14] and illustrated in Fig. 2. The rover state $\mathbf{x} = [\xi \ \eta \ \psi]^T$ encompasses the current longitudinal and lateral positions ξ and η , along with the heading angle ψ , expressed in a fixed initial frame. The target state from the provided trajectory is denoted by $\mathbf{x}_r$, and the error state $\tilde{\mathbf{x}}$ is calculated as

$$\tilde{\mathbf{x}} = \begin{bmatrix} \tilde{\xi} \\ \tilde{\eta} \\ \tilde{\psi} \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} (\mathbf{x} - \mathbf{x}_r). \quad (1)$$

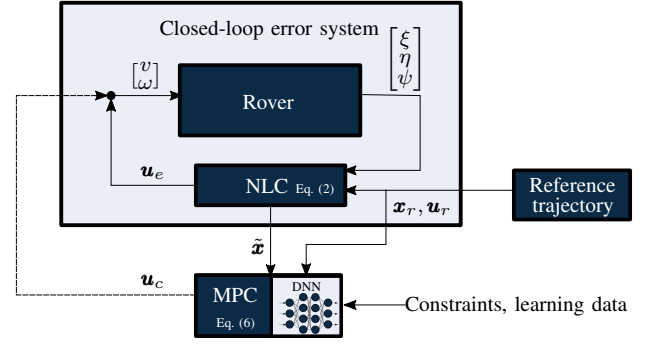


Fig. 2: Overview of trajectory tracking controller architecture from [14]. The overall controller consists of the nonlinear controller (NLC), which linearizes the system at a given trajectory with the reference states $\mathbf{x}_r$ and velocities $\mathbf{u}_r$, by computing the control input $\mathbf{u}_e$. An optional learning-based MPC receives the error state $\tilde{\mathbf{x}}$, constraints and learning data, and provides the additional control input $\mathbf{u}_c$.

The NLC provides the control input $\mathbf{u}_e$, which is composed of the commanded velocity and yaw rate v_e and ω_e :

$$\begin{aligned} v_e &= -K_\xi \tilde{\xi} + v_r \cos \tilde{\psi} \\ \omega_e &= -K_\psi \sin(\tilde{\psi}) + \omega_r - K_\eta v_r \tilde{\eta}. \end{aligned} \quad (2)$$

Here, v_r and ω_r represent the reference velocity and yaw rate of the target trajectory, while K_ξ , K_η , and K_ψ are scalar design parameters. The system can be linearized at the trajectory using the proposed NLC (2), which depends on the error state $\tilde{\mathbf{x}}$ from (1). This results in the following linear parameter-varying system (LPV):

$$\dot{\tilde{\mathbf{x}}} = \begin{bmatrix} -K_\xi & \omega_r & 0 \\ -\omega_r & 0 & v_r \\ 0 & -v_r K_\eta & -v_r K_\psi \end{bmatrix} \tilde{\mathbf{x}} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u}_c, \quad (3)$$

in matrix form

$$\dot{\tilde{\mathbf{x}}} = \mathbf{A}_d(K_\xi, K_\eta, K_\psi, v_r, \omega_r) \tilde{\mathbf{x}} + \mathbf{B}_d \mathbf{u}_c. \quad (4)$$

This approach enables trajectory following with only the NLC, with the option to enhance motion behavior through an additional controller input $\mathbf{u}_c$ as provided by the MPC. The simplicity and linearity of (3) ensures that the optimization problem remains straightforward and quickly solvable, allowing for efficient trajectory tracking.

IV. LEARNING-BASED ADAPTIVE MPC FOR TRAJECTORY TRACKING

A. Preliminaries

Due to the linearization of the system and unknowns such as slip effects, model uncertainties have to be considered. Therefore, a learning-based component using a *deep neural network* (DNN) is added to the MPC to compensate for the expected model error. In the proposed concept, the error of the state prediction over the prediction horizon is approximated using a DNN, based on a sequence of input feature vectors γ

further described in Sec. IV-C. This extends the LPV system dynamic from (3) with an error prediction to

$$\tilde{\mathbf{x}}_{k+1} = \mathbf{A}_d \tilde{\mathbf{x}}_k + \mathbf{B}_d \mathbf{u}_{c,k} + NN(\mathbf{\Gamma}_k), \quad (5)$$

with the discretized system matrices $\mathbf{A}_d \in \mathbb{R}^{3 \times 3}$ and $\mathbf{B}_d \in \mathbb{R}^{3 \times 2}$ (sample time 100ms) and the feature matrix $\mathbf{\Gamma}_k \in \mathbb{R}^{l \times n_\gamma}$ as input for the DNN. l denotes the number of used feature vector sequence and n_γ the number of used features in $\mathbf{\Gamma}_k$.

B. Outer-Loop adaptive Reference-Free Controller

To address additional performance goals beyond constraint satisfaction and leverage the advantages of learning-based model error prediction for the trajectory tracking task, a learning-based MPC is synthesized in the outer-loop to gain the optimal control input:

$$\begin{aligned} \min_{\tilde{\mathbf{x}}, \mathbf{u}} \quad & \tilde{\mathbf{x}}_N^T \mathbf{P} \tilde{\mathbf{x}}_N + \sum_{k=0}^{N-1} \tilde{\mathbf{x}}_k^T \mathbf{Q} \tilde{\mathbf{x}}_k + \mathbf{u}_{c,k}^T \mathbf{R} \mathbf{u}_{c,k} \\ \text{s.t.} \quad & \tilde{\mathbf{x}}_{k+1} = \mathbf{A}_d \tilde{\mathbf{x}}_k + \mathbf{B}_d \mathbf{u}_{c,k} + NN(\mathbf{\Gamma}_k) \\ & \tilde{\mathbf{x}}_0 = \tilde{\mathbf{x}}(t_0) \\ & \tilde{\mathbf{x}}_k \in \mathcal{X}, \mathbf{u}_k \in \mathcal{U}, k = 0, \dots, N-1 \\ & \tilde{\mathbf{x}}_N \in \mathcal{X}_f \end{aligned} \quad (6)$$

with a quadratic cost function and the weighting matrices $\mathbf{Q} = \mathbf{Q}^T \succeq 0$, $\mathbf{P} = \mathbf{P}^T \succeq 0$, and $\mathbf{R} = \mathbf{R}^T \succ 0$ and the prediction horizon N subject to the discretized nominal closed-loop LPV system dynamics (5), the state and input constraint sets $\mathcal{X}$ and $\mathcal{U}$, respectively. The terminal constraint set is $\mathcal{X}_f$.

C. Learning-based model error prediction

According to tests as performed in [10] and experience in previous projects, we designed the stacked feature vector of a dedicated time step s as input for the state error prediction as follows:

$$\gamma_s = [\tilde{\mathbf{x}}_s \ \tilde{\mathbf{x}}_i \ \mathbf{u}_{s,e} \ \mathbf{u}_{j,r} \ \boldsymbol{\sigma}_s]^T \in \mathbb{R}^{1 \times 59} \quad (7)$$

with the current state $\tilde{\mathbf{x}}_s \in \mathbb{R}^{1 \times 3}$, the stacked last predicted states $\tilde{\mathbf{x}}_i \in \mathbb{R}^{1 \times 30}$ of the time steps $i = s - N + 1, \dots, s - 1$, the NLC input $\mathbf{u}_{s,e} \in \mathbb{R}^{1 \times 2}$, the reference velocities $\mathbf{u}_{j,r} \in \mathbb{R}^{1 \times 20}$ with $j = s, \dots, s + N$ and the current IMU data $\boldsymbol{\sigma}_s = [\theta \ \phi \ F_\theta \ R_\theta]^T \in \mathbb{R}^{1 \times 4}$. The IMU data provided by the rover hardware consists of the body roll angle θ , the pitch angle ϕ and the roll angles for the front and rear bogie axles F_θ and R_θ . The feature matrix $\mathbf{\Gamma}_k$ builds the actual input for the DNN and consists of l sequences of the input feature vectors γ_s with $s = k - l, \dots, k$:

$$\mathbf{\Gamma}_k = [\gamma_{k-l} \ \dots \ \gamma_k] \in \mathbb{R}^{l \times 59}. \quad (8)$$

The function $NN(\mathbf{\Gamma}_k)$ then returns a sequence of predicted errors $\hat{\xi}$, $\hat{\eta}$ and $\hat{\psi}$ for the states $\tilde{\xi}$, $\tilde{\eta}$ and $\tilde{\psi}$ starting from $k + 1$ over the prediction horizon. Therefore, the DNN output dimension results in $\mathbb{R}^{N \times 3}$:

$$\begin{bmatrix} \Delta \hat{\xi}_{k+1} & \dots & \Delta \hat{\xi}_{k+N} \\ \Delta \hat{\eta}_{k+1} & \dots & \Delta \hat{\eta}_{k+N} \\ \Delta \hat{\phi}_{k+1} & \dots & \Delta \hat{\phi}_{k+N} \end{bmatrix} = NN(\mathbf{\Gamma}_k). \quad (9)$$

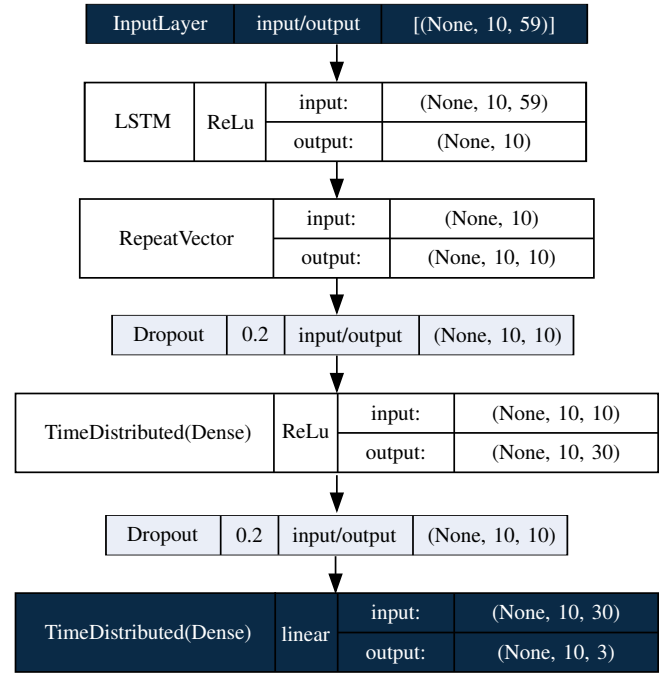


Fig. 3: Architecture of the model error prediction DNN, including the layer type, the activation functions and input/output dimensions.

With the prediction horizon $N = 10$ and the number of used feature sequences $l = 10$, we obtain the network architecture and its dimensions shown in Fig. 3. A major and most important part of the DNN architecture is the LSTM (Long Short-Term Memory) layer. It is a type of recurrent neural network (RNN) layer, which is tailored for handling sequences of data like time series or text. This type of neural network stands out from traditional RNNs because it is able to recognize long-term patterns in sequential data and avoids the vanishing gradient problem during training. For a further increased performance we use a combination of LSTM and Dense layers - This is a powerful architecture commonly used in sequence prediction tasks, such as time series forecasting, natural language processing, and speech recognition. The LSTM layer is utilized to capture temporal dependencies in sequential data, while the Dense layers are implemented to process the extracted features and make predictions based on the learned representations. In this architecture, the LSTM layer serves as the feature extractor, encoding the input sequences in $\mathbf{\Gamma}$ into a representation which is then passed to the Dense layers for further processing and prediction. The first Dense layer with ReLu (Rectified Linear Unit) activation functions can incorporate non-linearities and learn complex mappings between the LSTM output and the target variable. The second and last Dense Layer with linear activation functions reduces the non-linear distortions to the final output. To prevent overfitting during the training process, we also included Dropout layers as a regularization technique with a dropout rate of 20% in the architecture.

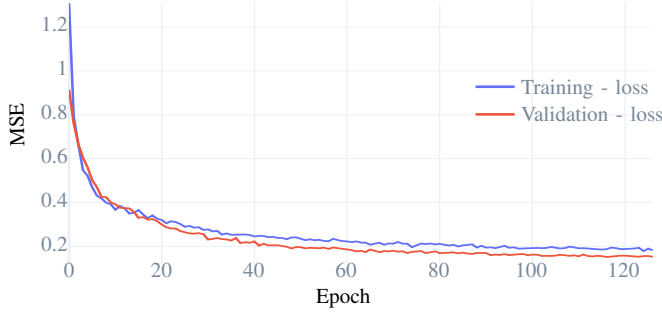


Fig. 4: Training history of the error prediction DNN with the MSE on the training data set (blue) and validation data set (red).

D. Data preparation and training

As preparation for the actual training and test campaign, we implemented a data pre-processing pipeline to ensure a consistent training process and collected initial datasets from the simulation and the real rover. In the pre-processing pipelines, standardization techniques are applied to ensure that all features have a similar scale, preventing any particular feature from dominating the model training process. Following feature standardization, the dataset is split into training and validation sets. The DNN model and the training is set up in the Keras¹ framework. Compiling the model with the built-in Adam optimizer and the *Mean squared error* (MSE) loss function, the training results in the history shown in Fig. 4. The training run shown in Fig. 4 was carried out with a recorded data set of the LRU in the DLR test facilities and achieved an R^2 -score² of 0.9149 on the training dataset and 0.8967 on the validation dataset. The R^2 -score is a statistical measure that indicates how well the regression model fits the ground-truth data. In the following, the inference results of the presented DNN are shown and compared, followed by the results of the adaptive MPC on the rover system.

V. TEST RESULTS AND FINDINGS

In this section, we start with a description of the initial test procedure involving various test drives conducted under defined conditions, as detailed in Algorithm 1. The individual test cases and configurations are subsequently compared based on their deviation from the specified trajectory within the same test setup. Throughout all experiments, the parameters of the NLC are selected as $K_\xi = 0.044731$, $K_\eta = 3.5293$, and $K_\psi = 1.2583$, according to stability design guidelines as referenced in [14]. The cost weights are chosen as $\mathbf{Q} = \mathbf{I}_3$ and $\mathbf{R} = 10\mathbf{I}_2$. In all cases, a constant speed of 0.3 m/s is commanded. The slope of the rear part of the testbed is 15 deg, a minimum of 3 laps are completed.

Following the initial phase, a ground truth test cycle is performed, employing the nominal MPC algorithm and executing a basic figure-of-eight trajectory. Subsequently, the DNN is

Algorithm 1 Testing procedure for the tracking performance, carried out in the PEL test facility

1. Ground truth drive with nominal MPC (Drive 1)
2. Train the DNN with training data
3. Open loop drive with the nominal MPC (Drive 2)
4. Closed loop drive with the adaptive MPC (Drive 3)
5. Train the DNN with a new dataset
6. Closed loop drive with newly trained DNN (Drive 4)

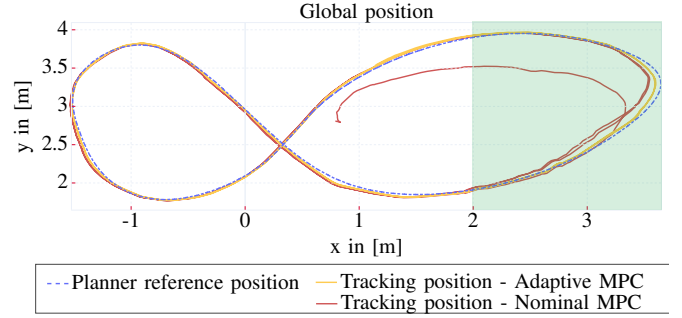


Fig. 5: Path of the Rover in the DLR test facility with the reference positions provided by the planning layer (blue) and both the measured positions of Drive 2 with the nominal MPC (red) and Drive 3 using the adaptive MPC (yellow). With the 15 degree sloped area marked in green.

trained using the acquired dataset from the initial test cycle. This training procedure is conducted on the Rover hardware as described in [10] and lasts approximately 3 minutes with a data size of 4 MB.

Afterwards, a new test drive is conducted with a new deformed figure-of-eight trajectory is employed, as illustrated in Fig. 5. In this test scenario, an adaptive MPC approach is adopted in an open-loop configuration. Here, the DNN is used to compute the state prediction errors, although without feedback to the optimization problem. The predictions generated by the DNN in this open loop setup can be inspected in [10]. They reveal that the MPC prediction error regarding ξ and η is predicted accurately both in amplitude and frequency. These results indicate that, by combining LSTM and Dense layers, the model can capture both short-term and long-term patterns in the sequential data and make accurate predictions. The proposed architecture strikes a balance between capturing temporal dependencies and incorporating high-level features, making it suitable for this specific prediction task.

It is important to underline that the prediction results perform Fig. 6 are promising, despite using a similar, but not identical trajectory. As the open-loop drive meets the expectations, the next step involves conducting a closed-loop drive utilizing Adaptive MPC (Drive 3). Fig. 5 compares the test drive with the nominal MPC (Drive 2) to the test drive with the adaptive MPC (Drive 3). It can be observed that although the rover starts Drive 2 with a larger deviation in the starting position, it gradually converges to the specified

¹<https://keras.io/>[Accessed 22.04.2024]

²https://keras.io/api/metrics/regression_metrics/[Accessed 23.07.2024]

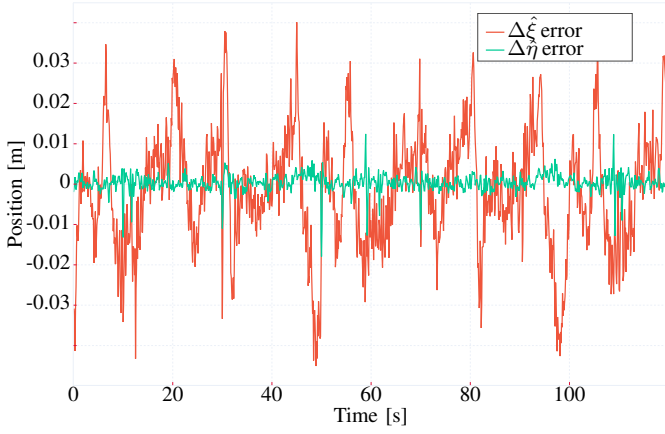
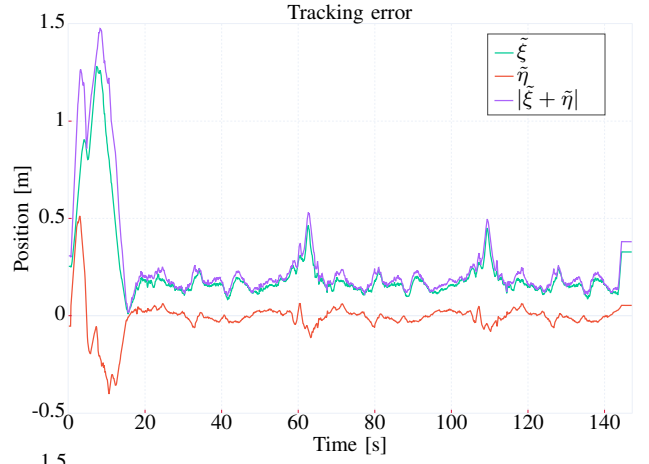


Fig. 6: Error of the DNN prediction $\Delta\hat{\xi}$ and $\Delta\hat{\eta}$ of the next time step $k + 1$ compared to the actual measured error. Tested in an open loop drive with the rover prototype in the test facility

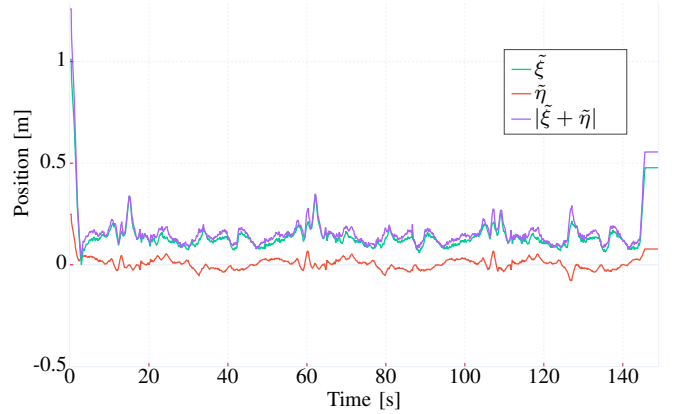
trajectory during the test drive. However, especially in the steep section of the test terrain, it exhibits a higher deviation compared to Drive 3. Fig. 7 compares the errors of the two test drives 2 and 3 where Fig. 7 (a) depicts a ride with the presented nominal MPC while Fig. 7 (b) represents the ride with the adaptive MPC. It is apparent that higher deviations occur at the start of the test drive due to the non-exact same starting position, making direct comparisons between these errors challenging. Therefore, the subsequent analysis focuses on errors after the initial alignment at approx. 20 s. It is evident that both lateral and longitudinal errors are reduced compared to the reference, due to the implementation of adaptive MPC. The improvement in the tracking performance is particularly noticeable during the incline of the test facility, see Fig. 7 (a) starting from 55 s and from 100 s. In the test ride with the nominal MPC, there is a maximum lateral deviation of 0.11 m and 0.46 m in the driving-direction during the run. In contrast during the test drive with the adaptive MPC, the errors are reduced to a maximum of 0.06 m in the y -direction and 0.34 m in the x -direction in the following laps.

The results of Drive 2 indicate that the nominal MPC is capable of following the trajectory well but there is potential for improvement in the MPC, especially in predicting the error in the rovers direction of travel. On the other hand, the adaptive MPC in Drive 3 performs well and reduces lateral deviation to a few centimeters. For a further comparison, the root mean squared error of the test run after the initial displacement is calculated and compared in Table I.

Table I reveals that not only the maximum deviations are reduced by the adaptive MPC, but the mean error during the test run is also significantly lower. In summary, the overall trajectory tracking error with the adaptive MPC improves by approx. 34 % in relation to the reference (ground truth) drive. For further improvement, the DNN training was carried out again before Drive 4. In this iteration, the recorded datasets



(a) Nominal MPC



(b) Adaptive MPC

Fig. 7: Comparison of the tracking performance of nominal MPC and adaptive MPC at a 15 deg slope. The plots show the tracking errors calculated during the test drive, with the error in driving direction ξ (green), the lateral error η (red) and the overall absolute error (purple) compared to the reference path.

TABLE I: Root mean squared error of the test drives 2-4 in m with the RMSE calculated from the measured positions by the tracking system. With the RMSE in x -direction (driving direction of the rover ξ), the error in y -direction (lateral deviation of the rover from the desired trajectory η) and the overall error (calculated from all absolute state errors).

	Ground truth	Closed loop	Closed loop trained
	Drive 2	Drive 3	Drive 4
RMSE ξ	0.3062	0.2075	0.1802
RMSE η	0.0920	0.0335	0.0305
RMSE Overall	0.3197	0.2102	0.2031

from Drive 1 and Drive 3 were used as training datasets. In the run with the updated DNN, it can be seen that the error is improved by 36% compared to the ground truth drive. Finally, computation times of the algorithms on the rover hardware are compared. The adaptive controller, including the optimization problem, averaged 31 ms per cycle during a test drive, with DNN inference taking an average of 90 μ s.

VI. DISCUSSION

The previous sections presented a control algorithm designed for trajectory tracking in uncertain terrain, with a particular focus on their computational simplicity and efficiency. Notably, the algorithms are computationally lightweight, making them suitable for on-board computations in space exploration scenarios. The integration of deep neural networks (DNNs) as frozen graphs further enhances computational efficiency, allowing for fast inference times and minimal hardware requirements, eliminating the need for dedicated GPUs.

We want to emphasize the small computation times required for both training and inference phases, as well as the small training data size necessary for effective learning. These characteristics are crucial for space exploration missions, where resource constraints are prevalent. Additionally, the algorithms' ability to operate efficiently without the need for high-performance computing resources ensures their practicality for deployment in real-world scenarios.

In terms of relevance to space exploration scenarios, the paper highlights the importance of the algorithms' performance in laboratory tests as a basis to their potential application in space missions. While the test facility setup may differ from actual space environments, the preliminary results provide valuable insights into algorithm effectiveness and robustness, as detailed in previous publications. Moreover, the utilization of a simulation allows for a fast development cycle, enabling unit tests and initial evaluation of DNN models before deployment on real rover hardware.

Potential improvements and lessons learned from the study include enhancing algorithm robustness through extensions such as tube-based MPC and integrating better online learning solutions. By addressing these aspects, future work aims to further optimize trajectory tracking algorithms for space exploration missions, ensuring adaptability and reliability in challenging environments.

VII. CONCLUSIONS

The present work presents promising advancements in the development of a learning-based adaptive MPC for trajectory tracking in uncertain terrain. Through the DeLeMIS project, innovative AI methods have been applied to enhance the autonomy of robotic vehicles, particularly in navigating challenging environments. The integration of machine learning algorithms, specifically LSTM and Dense layers, has shown promising results in improving trajectory tracking accuracy, reducing errors, and enhancing overall performance. The comprehensive test campaign conducted on real rover hardware

provides valuable insights into the capabilities and limitations of the proposed approach. Further iterations and refinements, including additional training and optimization of neural network architectures, demonstrate continuous improvement in trajectory tracking accuracy. Overall, the findings highlight the potential of learning-based control algorithms to address complex challenges in planetary exploration and pave the way for future advancements in autonomous navigation systems for space missions.

ACKNOWLEDGMENT

This work was funded from the ESA/ESTEC project DeLeMIS (ESA 4000136972/21/NL/AS).

REFERENCES

- [1] A. Ellery, *Planetary Rovers: Robotic Exploration of the Solar System*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016.
- [2] V. Gromov, A. Kemurdjian, A. Bogatchev, V. Koutcherenko, M. Malenkov, S. Matrossov, S. Vladykin, V. Petriga, and Y. Khakhanov, "Lunokhod 2-A retrospective Glance after 30 Years," in *EGS-AGU-EUG Joint Assembly*, 2003, p. 14528.
- [3] L. Meyer, M. Vayugundla, P. Kenny, M. Smíšek, J. Biele, A. Maturilli, M. G. Müller, W. Stürzl, M. J. Schuster, T. Bodenmüller, A. Wedler, and R. Triebel, "Testing for the MMX Rover Autonomous Navigation Experiment on Phobos," in *2023 IEEE Aerospace Conference*, Mar. 2023, pp. 1–19.
- [4] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, "Learning-Based Model Predictive Control: Toward Safe Learning in Control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, no. 1, pp. 269–296, 2020.
- [5] A. Mesbah, K. P. Wabersich, A. P. Schoellig, M. N. Zeilinger, S. Lucia, T. A. Badgwell, and J. A. Paulson, "Fusion of Machine Learning and MPC under Uncertainty: What Advances Are on the Horizon?" *Proceedings of the American Control Conference*, vol. 2022-June, no. 1, pp. 342–357, 2022.
- [6] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-based model predictive control for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, 2019.
- [7] J. Wang, M. T. H. Fader, and J. A. Marshall, "Learning-based model predictive control for improved mobile robot path following using Gaussian processes and feedback linearization," *Journal of Field Robotics*, vol. 40, no. 5, pp. 1014–1033, 2023.
- [8] U. Rosolia and F. Borrelli, "Learning How to Autonomously Race a Car: A Predictive Control Approach," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2713–2719, Nov. 2020.
- [9] P. K. Mishra, M. V. Gasparino, A. E. B. Velsasquez, and G. Chowdhary, "Deep model predictive control with stability guarantees," 2021.
- [10] K. Lakatos, N. Baldauf, and A. Turnwald, "Towards learning-based trajectory tracking control for a planetary exploration rover: Development and testing," in *1st International Conference on Space Robotics*, Juni 2024. [Online]. Available: <https://elib.dlr.de/205478/>
- [11] N. Baldauf, T. Lubiniecki, A. Turnwald, K. Lakatos, and N. Panagiotopoulos, "Learning-based motion control of a rover on unknown ground," in *12th International Conference on Guidance, Navigation & Control Systems (GNC)*. ESA, Jul. 2023.
- [12] A. Wedler, B. Rebele, J. Reill, M. Suppa, H. Hirschmüller, C. Brand, M. Schuster, B. Voderbauer, H. Gmeiner, A. Maier *et al.*, "LRU-lightweight rover unit," in *Proceedings of the 13th Symposium on Advanced Space Technologies in Robotics and Automation (ASTRA)*, 2015.
- [13] M. J. Schuster, S. G. Brunner, K. Bussmann *et al.*, "Towards Autonomous Planetary Exploration: The Lightweight Rover Unit (LRU), its Success in the SpaceBotCamp Challenge, and Beyond," *Journal of Intelligent & Robotic Systems (JINT)*, Nov. 2017.
- [14] N. Baldauf and A. Turnwald, "Iterative learning-based model predictive control for mobile robots in space applications," in *27th International Conference on Methods and Models in Automation and Robotics (MMAR)*, 2023, pp. 434–439.